

사업계획서

ScanOps

코드를 외부로 보내지 않는, AI 보안 취약점 자동 진단 SaaS

Self-hosted LLM Security Scanner — 단일코드 탐지 오탐률 6.0%·정확도 93.0% (Grok-3 동급) +
코드그래프 기반 멀티파일 taint 추적 정확도 100%, 코드 유출 0

버전 v6.0 · 2026 년 6 월

부산대학교 정보컴퓨터공학부 · 팀 ScanOps

팀장 김세한 · 팀원 전해은 · 이경윤 · 최효석

1. 사업 개요

ScanOps 는 GitHub 저장소나 웹 서비스 주소만 입력하면, 자체 파인튜닝한 경량 AI 모델이 소스코드의 보안 취약점을 자동으로 찾아 위험도(CVSS)와 수정 방법까지 제시하는 보안 진단 SaaS 다. 핵심 차별점은 분명하다. 분석에 쓰는 AI 모델을 외부 클라우드 API 에 의존하지 않고 자체 서버에서 직접 구동하기 때문에, 고객의 소스코드가 OpenAI·xAI 같은 외부로 단 한 줄도 나가지 않는다. v5 에서는 평가 관점을 '탐지율'에서 '오탐률(False Positive Rate)'로 확장해 오탐률 6%·정확도 93%(Grok-3 동급)를 검증했고, v6 에서는 여기에 코드 그래프 기반 멀티파일 데이터 흐름(taint) 추적 능력을 더해, 여러 파일에 걸친 취약점 판단에서도 100 케이스 기준 정확도 100% vs Grok-3 67%로 구조적 우위를 검증했다.

한 줄 정의

“전담 보안 인력이 없는 작은 개발팀도, 코드를 외부에 맡기지 않고 합리적 월 구독료로 쓸 수 있는 AI 보안 진단 도구”

구분	내용
제품	AI 기반 코드 보안 취약점 자동 진단 SaaS (SAST · DAST · GitHub Actions)
타겟	1 차: 보안을 우려하는 스타트업 CTO·소규모 개발팀 → 확장: 중소기업(SME)
수익모델	구독제 (Pro 29,900 원 / Max 99,000 원, 월) · 가입 시 DAST 1 회 무료 · Pro 7 일 무료체험 · 줄 추가 1 만줄당 5,000 원
핵심기술	Qwen2.5-Coder-1.5B QLoRA + Qdrant RAG + Hybrid Adjudication 오탐 필터 + 코드그래프 기반 taint 추적(v6 신규)
현재 단계	MVP 클라우드 배포, 단일코드 100 케이스(오탐 6%·정확도 93%) + 코드그래프 100 케이스(정확도 100%) 사내 벤치마크 검증, 예비창업

※ 사내 벤치마크 기준 초기 검증 결과(2026.6)

2. 문제 인식

2.1 코드는 폭증하는데, 보안은 따라가지 못한다

AI 코딩 도구의 확산으로 작성되는 코드의 양은 폭발적으로 늘고 있지만, 그 코드를 검증할 보안 역량은 같은 속도로 늘지 않는다. 보안 취약점은 배포 전에 잡으면 수정 비용이 낮지만, 운영 단계에서 터지면 데이터 유출·서비스 중단으로 이어진다. 그럼에도 작은 팀일수록 보안 점검은 '나중 일'로 밀린다. 이유는 단순하다 — 마땅한 도구가 없기 때문이다.

2.2 타깃 고객의 3대 페인포인트

- ① 전담 보안 인력·예산이 없다 — 스타트업·소규모 팀은 보안 엔지니어를 둘 여력이 없다. 개발자가 보안까지 겸하지만 전문가가 아니라 취약점을 놓친다.
- ② 기존 SAST 도구는 비싸고 무겁다 — 스파로우 등 국내 상용 정적분석 도구는 엔터프라이즈 견적·연단위 라이선스 중심이라 도입 장벽이 높다.
- ③ 클라우드 AI 에는 민감한 코드를 맡기기 어렵다 — ChatGPT·Claude 에 코드를 붙여넣는 방식은 편하지만 핵심 소스코드를 외부 서버로 전송한다는 근본적 모순이 있다.

정리하면, 작은 팀은 '싸고 가볍고 코드가 새지 않는' 보안 진단 도구를 원하지만 시장에는 그 셋을 동시에 만족하는 선택지가 없다. ScanOps 는 정확히 이 빈 공간을 겨냥한다.

3. 솔루션: ScanOps

3.1 작동 원리

사용자가 GitHub 주소나 웹 URL 을 입력하면 백엔드(Spring Boot)가 코드를 받아 분석 서버(FastAPI)로 보내고, 자체 구동하는 AI 모델(Ollama + Qwen2.5-Coder-1.5B, QLoRA 파인튜닝)이 취약점을 분석한다. 필요 시 벡터 DB(Qdrant, 약 792 건 CVE 적재)에서 유사 취약점 사례를 검색해 정확도를 보강하고, 여러 파일에 걸친 취약점은 코드 그래프 엔진이 실제 데이터 흐름을 추적해 근거를 보강한다. 분석에 쓰인 소스코드는 메모리에서만 처리되고 즉시 폐기되며, 결과만 저장된다.

i 현재 상태 코드 그래프 추론 로직은 구현·검증 완료(아래 3.5 절 100 케이스 벤치마크 참조)했으나, 현재는 Neo4j 그래프 DB 가 아닌 인메모리 엔진으로 동작한다. Neo4j 연동(그래프 DB 저장·시각화)과 프론트엔드 그래프 화면은 로드맵(7.2 절) 항목으로, 아직 연동 전이다.

3.2 핵심 기술

- 자체 파인튜닝 모델: Qwen2.5-Coder-1.5B 에 QLoRA 로 보안 특화 학습. 양자화 후 약 1GB 로 경량화해 GPU 없이 CPU 서버에서도 구동.
- RAG(검색 증강): NVD CVE 를 벡터 검색해 '그럴듯한 추측'이 아닌 실제 취약점 사례에 근거한 진단 제공. 최신 NVD CVE 를 실시간 참조해 학습 컷오프가 있는 범용 LLM 이 모르는 신규 취약점에 강하다.
- Hybrid Adjudication 오탐 필터: 1 차 탐지 결과를 [정적 mitigation 분석 → SAFE 보정] + [소형 LLM 판정]의 2 단계 게이트로 검증해, 안전한 코드를 취약으로 잘못 경고하는 오탐을 대폭 제거. v4 raw 모델의 오탐률 100% → 현재 6%로 개선.
- 코드 그래프 기반 taint 추적(v6 신규): 파일 간 import·prop·별칭 관계를 그래프로 추출해, 1 차 LLM 판단이 정적 자원에서 온 오탐인지 실제 사용자 입력이 위험 sink 까지 도달하는 진짜 취약점인지 구조적으로 검증한다.
- CVSS 위험도 산출: 취약점마다 심각도 점수를 매겨 '무엇부터 고쳐야 하는지' 우선순위를 제시.

3.3 기술 성과 ① — 단일코드 탐지·오탐률 벤치마크 (100 케이스)

벤치마크 설계

- 100 개 케이스 = 양성(취약) 50 + 음성(안전) 50. 양성은 2026 년 5~6 월 NVD 신규 공개 CVE 패턴 기반(범용 LLM 학습 컷오프 이후 → 암기 불가).

- 음성 50 개는 parameterized 쿼리·출력 이스케이프·authz·상수시간 비교 등 mitigation 적용 코드 및 순수 로직 → 오탐(False Positive) 측정용.
- 동일 파이프라인에 LLM 코어만 교체해 ScanOps vs Grok-3(xAI)를 1:1 공정 비교.

성능 비교 결과

시스템	탐지율	오탐률	정밀도	정확도	F1	응답
ScanOps v4-raw (필터 없음)	100%	100%	50%	50%	66.7	—
ScanOps (하이브리드)	92.0%	6.0%	93.9%	93.0%	92.9	0.2s
Grok-3-mini (xAI)	86.0%	0.0%	100.0%	93.0%	92.5	2.14s

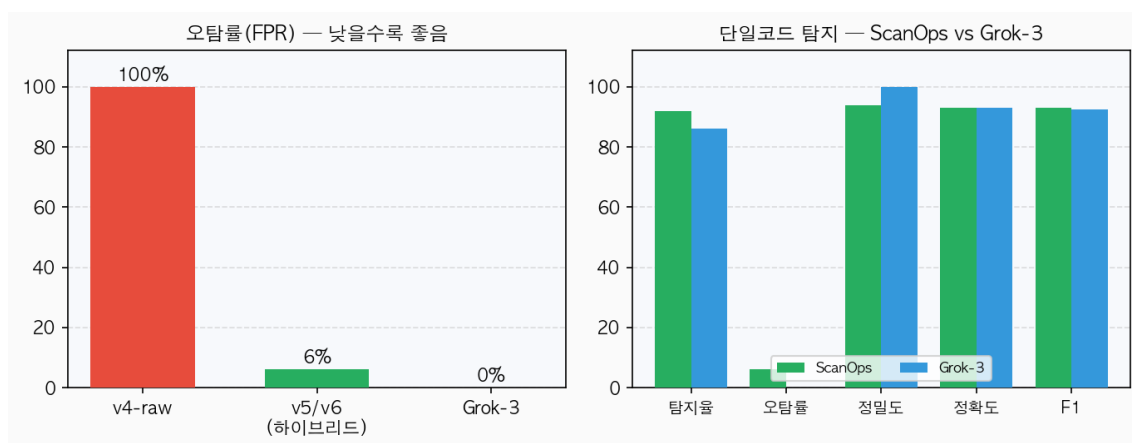


그림 1. 단일코드 오탐률 벤치마크 — v4-raw 100% → 하이브리드 6%, 정확도는 Grok-3 와 동일(93%)

- 정확도 동일(둘 다 93%)·F1 동등(92.9 vs 92.5) — 종합 성능은 상용 모델과 동급.
- ScanOps 는 취약점을 4 건 더 탐지(46 vs 43)하는 '안전 중심', Grok 은 오탐 0 의 '정밀 중심'. 흥미롭게도 Grok 은 권한·정책 계열(Missing AuthZ/AuthN, Hardcoded, Rate Limit 등) 7 건을 놓쳤고 ScanOps 가 이를 탐지했다.
- 응답 속도 약 10 배(0.2s vs 2.14s), 외부 API 호출 0 — 자체 모델만으로 달성.

※ 본 결과는 사내 벤치마크 100 케이스 기준 초기 검증 결과이며, 실제 프로덕션 성능은 베타 사용자 데이터로 지속 개선한다.

3.4 심사 포인트 — '범용 LLM 은 놓치고 ScanOps 는 잡은' 신규 CVE 사례

심사위원과 잠재 고객이 가장 궁금해하는 것은 '보안 전용으로 파인튜닝했다'는 설명이 아니라, 'Claude Code·Codex 같은 유명 에이전트나 유료 플랜에 코드를 넣어 검사하는 것보다 실제로 더 나은가'이다(인터뷰 §10 참조). 이를 직접 보여주기 위해, 범용 LLM 의 학습 컷오프 이후인 2026 년 5~6 월에 공개된 신규 CVE 패턴에서, 동일 코드를 ScanOps 와 범용 프런티어 모델(Grok-3)에

똑같이 입력해 비교했다. 결과는 명확하다 — 아래 취약점들을 ScanOps 는 '취약'으로 탐지했고, Grok-3 는 모두 '안전(SAFE)'으로 잘못 판정해 놓쳤다.

대표 사례 (동일 입력, 상반된 판정)

신규 CVE (공개일)	입력 코드	실제 취약점	ScanOps	Grok-3
CVE-2026-44754 (2026-06-09, SAP)	<code>app.post("/api/replicate", (req,res)=>{ replicate(req.body); res.sendStatus(200); });</code>	Missing Authorization (CWE-862) — 인증/권한 검사 없이 민감 작업 실행	취약 탐지 ✓	SAFE (미탐) ✕
CVE-2026-11572 (2026-06, degit)	<code>function merge(t,s){ for(const k in s){ t[k]=s[k]; } } merge({}, JSON.parse(req.body));</code>	Prototype Pollution (CWE-1321) — 신뢰 불가 입력의 재귀 병합	취약 탐지 ✓	SAFE (미탐) ✕
CVE-2026-21404 (2026-06-04, NAVTOR)	<code>String SOAP_USER="svc"; String SOAP_PW="P@ssw0rd!"; auth(SOAP_USER, SOAP_PW);</code>	Hardcoded Credentials (CWE-798) — 소스에 자격증명 하드코딩	취약 탐지 ✓	SAFE (미탐) ✕

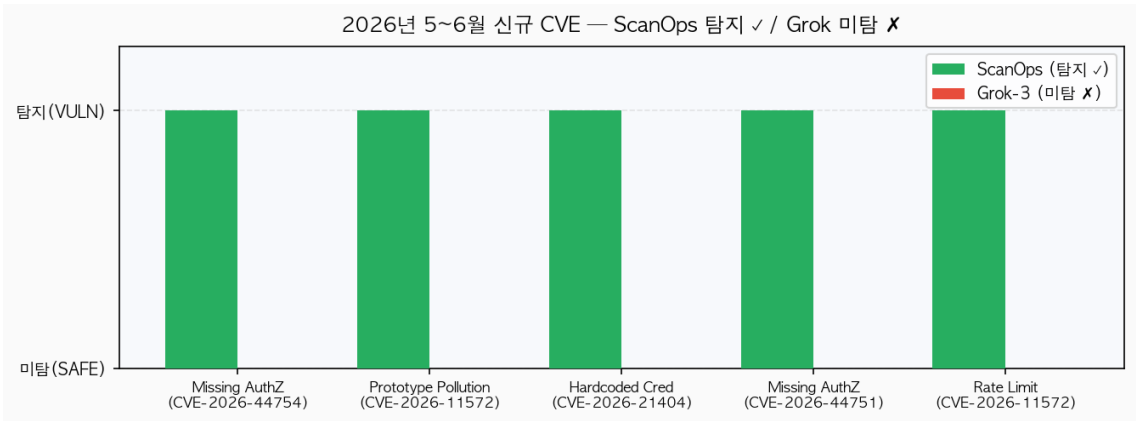


그림 2. 2026 년 5~6 월 신규 CVE 5 건 — ScanOps 전건 탐지(✓), Grok 전건 미탐(✕)

왜 이런 차이가 나는가.

- 신규성: 이 CVE 들은 범용 LLM 의 학습 컷오프 이후 공개됐다. Claude Code·Codex·Grok 등 유명 에이전트는 모두 동일한 범용 LLM·학습 컷오프 한계를 공유하므로 '본 적 없는' 신규 패턴에 약하다. 본 벤치마크는 직접 실행 가능한 Grok-3 로 그 한계를 대표 검증했다.
- 권한·정책 계열: 위 사례들은 코드 자체는 평범해 보이지만 '있어야 할 인증·권한·검증이 없음'이 취약점이다. 범용 LLM 은 이런 'negative space(없는 것)'를 놓치는 경향이 강하다.

- ScanOps 의 강점: 보안 특화 파인튜닝 + 최신 NVD CVE 를 실시간 검색(RAG)하는 구조라, '오늘 새로 올라온' 취약점 패턴도 근거를 가지고 탐지한다.

3.5 기술 성과 ② — 코드그래프 기반 멀티파일 taint 추적 (v6 신규, 100 케이스)

위 3.3 절은 '단일 코드 스니펫'에서의 탐지 정확도를 본다. 그러나 실제 코드는 여러 파일에 걸쳐 데이터가 흐른다 — 예를 들어 부모 컴포넌트가 정적 이미지(import)를 prop 으로 넘기면 안전하지만, 동일한 코드 패턴이라도 URL 파라미터 같은 사용자 입력을 넘기면 실제 XSS-SSRF 위험이다. 이 둘을 텍스트만 보고 구분하는 것은 프런티어 모델에게도 어렵다. ScanOps 는 파일 간 import·prop·별칭 관계를 그래프로 추출해 이 구분을 구조적으로 해결한다.

벤치마크 설계

- 100 개 케이스 = 2026 년 5~6 월 NVD 실제 XSS(CWE-79)·SSRF(CWE-918) CVE 25 개씩 기반 50 개 + sink 종류·prop 전달 깊이(0~2 단계)·별칭 체인 조합 50 개.
- ScanOps 는 실제 API 서버가 사용하는 그래프 판정 로직을 그대로 사용, Grok-3-mini 는 그래프 없이 동일한 멀티파일 코드를 텍스트로만 보고 VULNERABLE/SAFE 판정.

성능 비교 결과

시스템	정확도	Recall(탐지율)	Specificity(오탐방지)	FP	FN
ScanOps (코드그래프)	100.0%	100.0%	100.0%	0 건	0 건
Grok-3-mini (코드만)	67.0%	35.3%	100.0%	0 건	33 건

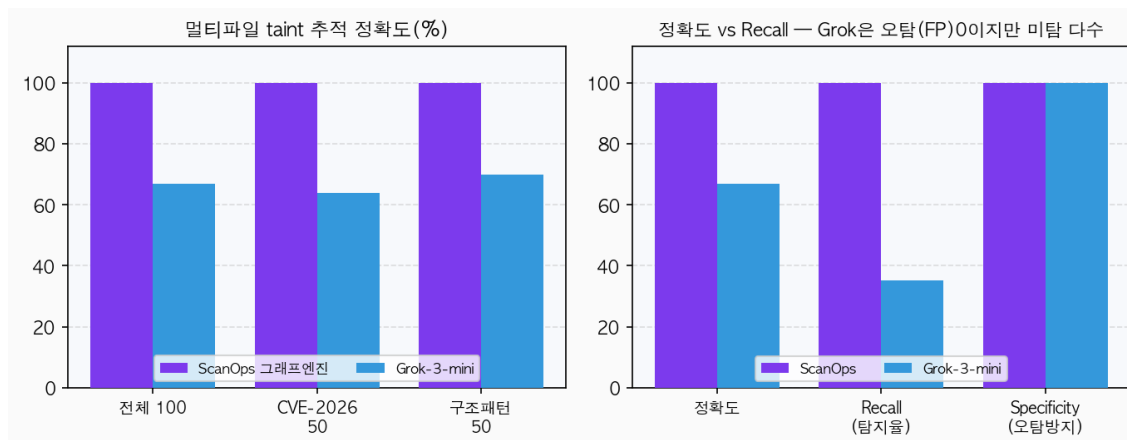


그림 3. 코드그래프 100 케이스 — ScanOps 100% vs Grok-3-mini 67.0%(recall 35.3%)

- Grok 은 오탐(FP) 0 건으로 보수적이지만, 사용자 입력이 prop·별칭을 거쳐 실제 위험 sink(fetch, axios, img src 등)에 도달하는 51 개 중 33 개를 '확신 없음 → SAFE'로 놓쳤다(recall 35.3%).

- ScanOps 그래프 엔진은 hop 깊이(0~2 단계)·별칭 체인과 무관하게 100 케이스 전부 정답을 맞췄다 — 멀티파일 taint 추적은 모델 크기 문제가 아니라 그래프 기반 아키텍처의 차이임을 확인했다.
- 이 결과는 텍스트만 보는 범용 LLM 이 구조적으로 약한 지점(멀티파일 데이터 흐름 증명)에서 ScanOps 가 명확한 우위를 가진다는 것을 보여준다.

i 현재 상태 본 벤치마크는 그래프 판정 로직 자체의 정확도를 검증한 것이며, 위 3.1 절에서 밝힌 대로 현재 Neo4j 그래프 DB·프론트 시각화는 아직 연동 전(로드맵 항목)이다. 판정 정확도와 데이터베이스·시각화 연동은 별개이며, 정확도는 인메모리 엔진으로도 동일하게 보장된다.

4. 차별성 및 경쟁 우위

4.1 경쟁 포지셔닝

경쟁 환경을 '도입 비용·진입장벽'과 '코드 프라이버시(외부 전송 여부)' 두 축으로 보면, ScanOps 는 '저가·셀프서브이면서 코드를 외부로 보내지 않는' 영역을 사실상 단독 점유한다. 엔터프라이즈 도구(스패로우)는 강력하지만 비싸고 무겁고, 클라우드 SaaS(Snyk·Semgrep)나 범용 AI(ChatGPT·Claude·Grok)는 편하지만 코드를 외부로 전송한다.

4.2 경쟁사 비교

① 가격 비교

도구	가격	비고
ScanOps	월 29,900 원~	Pro/Max 플랜, 줄 기반 과금
Snyk	월 \$25~57/contributor	팀 규모에 따라 급증
Semgrep	월 \$40/contributor~	커스텀 룰 작성 필요
GitHub Advanced Security	무료(public)/유료(private, per committer)	GitHub 전용
스패로우	엔터프라이즈 견적	연단위 라이선스

② 코드 프라이버시

도구	코드 전송 여부	비고
ScanOps	전송 안 함	자체 서버 처리·즉시 폐기
Snyk / Semgrep(상용)	클라우드 전송	외부 서버 분석
범용 AI(ChatGPT·Claude·Grok)	클라우드 전송	프롬프트로 코드 유출
스패로우	온프레미스	자체 설치

③ AI 보안 특화 / 멀티파일 분석

도구	방식	비고
ScanOps	보안 파인튜닝 + 하이브리드 오탐 필터 + 코드그래프 taint 추적	단일코드 오탐률 6% · 멀티파일 정확도 100%(사내 벤치마크)
Snyk	룰 + 일부 AI(DeepCode)	클라우드
Semgrep	룰 기반	커스텀 룰 강점
범용 AI	범용 LLM, 텍스트만으로 판단	멀티파일 데이터 흐름 증명에 구조적으로 약함(자체 벤치마크 recall 35.3%)

4.3 ScanOps 만의 네 가지 무기

- 코드 유출 0 — 자체 구동 모델로 소스코드가 외부로 나가지 않으며, 메모리 처리 후 즉시 폐기하고 삭제 로그로 증명한다.
- 동등 성능 + 합리적 가격 — 단일코드 100 케이스에서 상용 모델과 동일한 정확도(93%)를, 월 2.9~9.9 만원 셀프서브로 제공.
- 오탐 관리 + 신규 CVE 강점 — 하이브리드 필터로 오탐 6%, NVD RAG 로 학습 컷오프 이후 신규 취약점까지 커버.
- 멀티파일 taint 추적(v6 신규) — 코드그래프로 파일 간 데이터 흐름을 추적해, 범용 LLM 이 구조적으로 약한 멀티파일 오탐·미탐 구간(정확도 100% vs Grok 67%)에서 명확한 우위.

4.4 경쟁 우위의 방어 (Moat)

대기업(GitHub-Snyk)이 유사 기능을 무료화할 경우의 대응:

- 커스텀 룰: 소규모 팀별 스택(예: React+Firebase)에 특화된 맞춤 탐지 룰 제공.
- 빠른 대응: 고객 요청 → 모델 재학습 → 배포까지 1 주일 이내. 대기업은 의사결정이 느리다.
- 한국어 특화(향후): 한국어 취약점 설명 + 국내 법규(개인정보보호법) 맞춤 리포트.

5. 시장 분석

5.1 시장 성장성

글로벌 애플리케이션 보안(AppSec) 시장은 2025 년 약 USD 106.5 억(약 14.8 조원)에서 2033 년 약 USD 420 억 규모로 연평균 18.8% 성장이 전망된다. 클라우드 전환과 AI 코딩 도구 확산으로 'SAST/DAST' 수요가 빠르게 늘고 있으며, 아시아·태평양이 가장 높은 성장률을 보인다. 국내 정보보호산업도 2024 년 18.6 조원으로 전년 대비 10.5% 성장했다.

5.2 TAM / SAM / SOM

구분	정의	규모(추정)
TAM	글로벌 애플리케이션 보안 시장 전체	약 14.8 조원
SAM	클라우드형 코드보안 SaaS · SME/소규모팀 세그먼트	약 8,800 억원
SOM	3 년 내 도달 가능한 국내 beachhead(스타트업·소규모팀, bottom-up)	약 15 억원

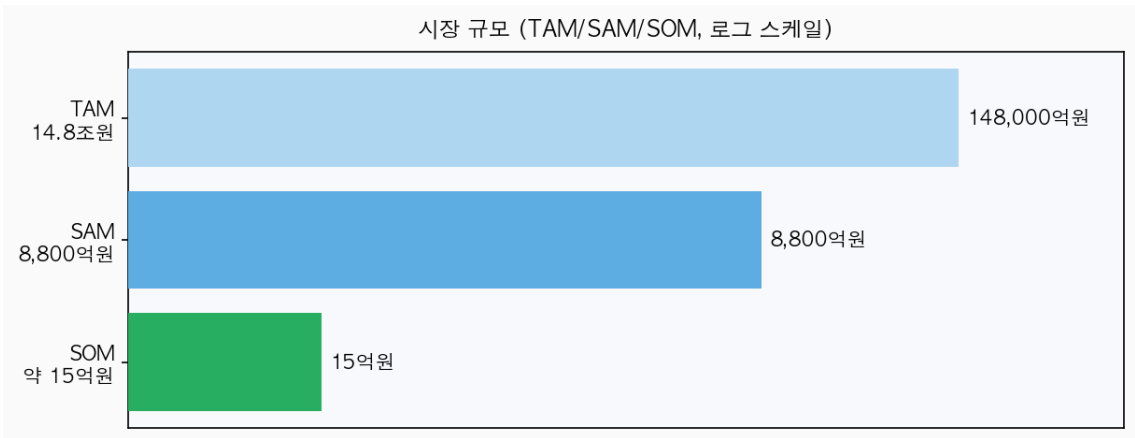


그림4. 시장 규모 추정 (TAM: Grand View Research 2026 · SAM/SOM 가정 기반)

출처 및 산정 근거. TAM 은 Grand View Research, 'Application Security Market Size, Industry Report 2033'(2026)의 2025 년 세계 애플리케이션 보안 시장 USD 106.5 억(약 14.8 조원, 환율 1,390 원/USD)·연평균 18.8% 전망을 인용했다. SAM 은 클라우드형 SAST/DAST 이면서 SME/소규모팀 세그먼트를 약 6%로 가정해 약 8,800 억원으로 top-down 추정했다. SOM 은 bottom-up 으로, 3 년 내 도달 가능한 국내 스타트업·소규모 팀 유료 계정과 상향된 ARPU(Pro·Max 혼합)를 반영해 약 15 억원 규모로 보수적으로 산정했다. (TAM/SAM/SOM 정의·규모는 이전 버전 대비 변동 없음. 모든 추정치는 예비창업 단계의 가정 기반이며 베타 데이터로 정밀화한다.)

6. 비즈니스 모델

6.1 구독 플랜

플랜	가격(월)	DAST(웹)	GitHub App(액션)	SAST(레포)
회원가입 시	0 원	1 회 무료	X	X
Pro	29,900 원	월 5 회	월 5 만 줄	월 10 만 줄
Max	99,000 원	월 30 회	월 30 만 줄	월 50 만 줄

추가 결제(한도 초과 시, Pro-Max 공통): SAST-GitHub App 1 만 줄당 5,000 원 · DAST 웹 스캔 3 회당 5,000 원 ·

Pro 는 가입 후 7 일 무료체험 후 자동 결제 전환

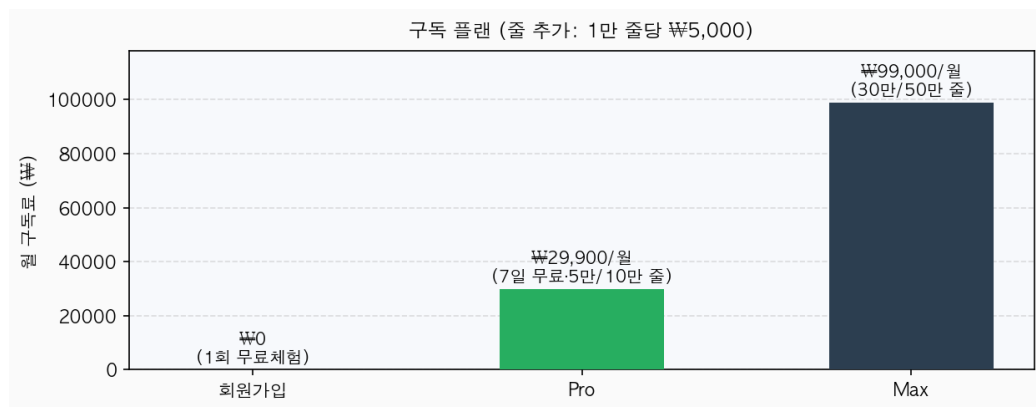


그림 5. 구독 플랜 — 회원가입(무료체험) / Pro / Max

무료 구독 플랜은 두지 않는다. 대신 회원가입 시 웹 URL DAST 를 1 회 무료 제공하고(PLG), Pro 는 7 일 무료체험 후 결제로 전환되게 해 진입 마찰을 낮춘다. DAST 는 외부 웹을 능동 스캔해 1 건당 자원이 커 월 횟수 상한(Pro 5 회·Max 30 회)을 둔다. SAST-GitHub App 은 코드량(LOC) 한도로 통제하며(Pro: 액션 5 만·SAST 10 만 줄 / Max: 액션 30 만·SAST 50 만 줄), 한도를 넘으면 SAST-GitHub App 은 1 만 줄당 5,000 원, DAST 웹 스캔은 3 회당 5,000 원으로 종량 추가 구매한다. LOC·횟수 한도와 추가 구매는 서버 비용을 통제하는 안전장치이자 매출의 상방을 여는 장치다.

가격 책정 근거. 인터뷰(\$10)에서 확인된 타깃(스타트업 CTO·보안 우려 개발팀)은 개발도구에 월 14 만원 이상을 쓰고 'CTO 는 월 10 만원대 지출을 고민하지 않는' 가격 민감도가 낮은 층이다. 저가 박리다매(월 1 만원 이하)는 다수 무료·이탈 사용자를 유인해 서버 부담만 키우는 반면, 보안의 가치를 아는 소수 유료 고객에게 제값을 받는 편이 단위경제와 손익분기에 유리하다.

6.2 단위 경제 및 손익분기 (Unit Economics)

- 변동비는 매우 낮다 — 스캔 1 건당 실제 비용은 DAST 약 1 원, SAST(5 만 줄) 약 20 원 수준. 약 1GB 경량 모델이라 GPU 없이 CPU 로 구동.
- 서버 고정비(Railway, 모델+Qdrant+API 상시 가동) 약 월 50 만원. 결제 수수료는 매출의 약 3%(토스/스트라이프) 가정.
- 인프라 손익분기: 고정비 50 만원 기준, 수수료 차감 후 Pro(약 29,000 원) 약 18 명 또는 Max(약 96,000 원) 약 6 명이면 서버 고정비를 회수한다.
- 마케팅 포함 손익분기: 개발자 유튜버 협찬·광고대행 비용을 반영한 고객획득비용(CAC)을 1 인당 약 5 만~15 만원으로 보수적 가정. 타겟의 낮은 가격 민감도와 높은 잔존(월 이탈 ~5% 가정 시 평균 20 개월 구독)으로 Max 기준 LTV $\approx$ 약 180 만원이 되어 LTV/CAC $\approx$ 12 배 이상. 즉 CAC·수수료·서버를 모두 반영해도 유료 전환 수십 명 규모에서 흑자 구조에 진입한다.

i 현재 상태 인프라는 Railway 만 실제로 사용 중이다(Dockerfile-railway.toml-docker-compose 기준 확인). 코드그래프용 Neo4j(Aura 등)는 아직 계정·인스턴스가 없어 별도 고정비에 포함되지 않았으며, 연동 시 고정비에 소액 추가될 수 있다(Aura 무료 티어로 시작 가능).

7. 사업화 전략 및 로드맵

7.1 Go-To-Market: CTO 타겟 PLG → SME → 글로벌

초기 타겟은 '인디 개발자 다수'가 아니라 '보안을 우려하는 스타트업 CTO'다. 인터뷰(\$10)에서 드러났듯 스타트업을 시작하는 개발자가 가장 우려하는 지점이 바로 보안이며, 이들은 월 10 만원대 지출을 고민하지 않는다. Reddit-GitHub Marketplace 는 전환율을 기대하기 어려워 주력 채널에서 제외하고, 다음 채널 전략으로 전환한다.

주력 채널 — 개발자 유튜버 협찬

- 개발자 유튜버(백엔드·DevOps·스타트업 채널)와의 협찬/리뷰로 CTO·시니어 개발자에게 직접 도달. 인스타 광고는 타겟 적합도가 낮아 유료 광고에서 제외하고 유튜브에 집중.
- '신규 CVE 30 초 분석', 'ScanOps 로 내 레포 스캔해보기' 등 제품 강점을 짧은 데모 영상으로 전달.

브랜드 채널 — 인스타그램 'ScanOps Security Report'

- 주간 단위로 발견·분석한 취약점을 잡지/홍보물 형식의 카드뉴스로 발행하는 인스타 계정을 운영. (광고가 아니라 콘텐츠로 계정 자체를 키운다.)
- '이번 주 발견된 위험 취약점' 업적을 꾸준히 알리며 전문성·신뢰를 축적 → 팔로워가 잠재 고객으로 유입되는 오가닉 획득 채널.

유료 전환 메커니즘

- 매 PR 마다 GitHub App 이 PR 코멘트로 취약점 알림 → 습관화 → Pro 7 일 무료체험 → 자동 결제 전환.
- 랜딩페이지에 실제 탐지 사례·UI 스크린샷·CTO 후기 노출(인터뷰에서 '실제 탐지 예시가 와닿으면 관심이 생긴다' 확인).

7.2 단계별 로드맵

단계	기간	핵심 과제
0. 그래프 인프라	즉시~3 개월	Neo4j(Aura) 연동, 백엔드 DTO 에 graph_evidence-kg_risk_score 필드 추가, 프론트 코드그래프 시각화 UI
1. 토대	0~3 개월	사용자 인증·계정, 모든 스캔을 user_id 에 연결, XLS 리포트·NVD 알림 기능
2. 게이트	3~6 개월	플랜별 사용량(LOC) 한도·줄 추가 구매, 레포/도메인 소유권 인증
3. 계정 UI	6~9 개월	마이페이지, 스캔 기록·삭제, 사용량 미터

4. 수익화	9~12 개월	결제 연동(토스/스트라이프), Pro 7 일 무료체험 전환 퍼널
5. 차별화	12~18 개월	경쟁사 대비 오답률·멀티파일 정확도 벤치마크 공개, 글로벌 진출 준비

7.3 3 개년 추정 손익

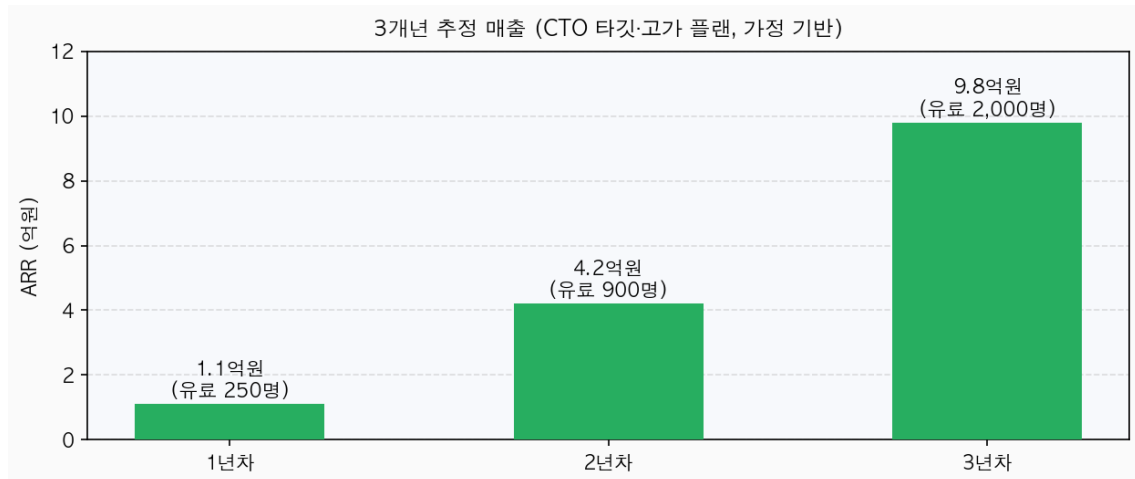


그림 6.3 개년 매출 고객 추정 (CTO 타겟·고가 플랜, 가정 기반)

타겟을 고가치 CTO 로 좁히고 Pro 29,900 원·Max 99,000 원으로 ARPU 를 끌어올렸다. 1 년차 ARR 약 1.1 억원(유료 250 명), 2 년차 약 4.2 억원(900 명), 3 년차 약 9.8 억원(2,000 명)을 목표한다. 변동비가 극히 낮고 인프라 손익분기가 Pro 약 18 명 수준이라, 유료 전환이 본격화되는 1 년차 후반~2 년차부터 흑자 전환이 가능한 구조다. (가정 기반 추정치, 베타 데이터로 정밀화)

8. 팀 구성

ScanOps 는 부산대학교 재학생 4 인으로 구성되며, AI 모델·백엔드·인프라·보안·프론트·UI/UX·마케팅까지 SaaS 운영에 필요한 역량을 자체 보유한다.

팀원	역할
김세한 (팀장)	AI 모델 · 프론트엔드 · 총괄
전혜은	백엔드 · 인프라
이경윤	AI 모델 · 보안
최효석	UI/UX · 마케팅 · QA

9. 기대효과 및 결론

ScanOps 는 '작은 팀도 코드를 외부에 맡기지 않고 합리적 가격에 쓰는 AI 보안 진단 도구'라는, 시장에 비어 있던 자리를 정확히 겨냥한다. 자체 모델로 단일코드 100 케이스에서 오탐률 6%·정확도 93%(상용 Grok-3 동급)를 달성해 '탐지율'을 넘어 '오탐 관리'까지 검증했고, v6 에서는 코드그래프 기반 멀티파일 taint 추적으로 정확도 100%(Grok-3 67%)를 추가 검증해 범용 LLM 이 구조적으로 약한 구간에서도 명확한 우위를 확인했다. 기술적 실현 가능성(이미 구현됨), 명확한 차별성(코드 비전송 + 오탐 관리 + 멀티파일 정확도 + 합리적 가격), 성장하는 시장(연 18.8%)을 모두 충족하는 사업이다.

핵심 메시지

“보안을 위해 코드를 외부에 넘기지 않아도 된다. ScanOps 는 자체 AI 와 코드그래프로, 단일 코드부터 여러 파일에 걸친 취약점까지 상용 모델과 동등하거나 더 나은 정확도를, 작은 팀이 감당할 가격에 제공한다.”

10. 고객 검증 및 향후 계획

10.1 초기 인터뷰 결과

현재까지 총 2 인 인터뷰를 완료했다. ① 스타트업 SW 개발자(7 문항 심층 인터뷰)와 ② 실사용·제품화 관점의 스타트업 개발자다. 주요 내용과 시사점은 다음과 같다.

인터뷰 ① 스타트업 SW 개발자 (7 문항 심층)

- 보안 지식: 스스로 10 점 만점에 2~3 점. 개인정보 비식별화 정도만 처리해 왔고 공격·방어 지식은 부족.
- 보안 점검: 필요성은 인지하나 받은 적 없음. '빠른 출시'가 우선이었고, 비개발 팀원 설득 비용 대비 가치가 낮게 느껴졌으며 '내 돈'으로는 쓰고 싶지 않음(회사가 낸다면 100~200 만원 이내 의향).
- 도구 지출: 월 약 14 만원(Cursor 3 만·Claude Code 3 만·GPT 3 만·AWS 4~5 만). 가장 비싼 건 AWS. 개당 3 만원대 지출이 일상적.
- AI 보안 도구 의향: '관련 오픈소스나 AI 가 있으면 꼭 써보고 싶다.' 과거 OWASP SAST 도구(Noir)를 시도했으나 기획 변경으로 홀딩. '정식 배포 전 기본 보안 점검은 무조건 필요'.
- 프라이버시 우려: '보안 리포트가 서버로 가면서 정보가 같이 새지 않을까'를 우려 → 그러나 실제 사용 후 '개 좋은데?! 공격 위험도·방어방침·참고문서가 같이 나와 좋다, 개인 프로젝트 보안 학습에도 도움'이라며 만족.

인터뷰 ② 스타트업 개발자 (실사용·제품화 관점)

- 초기 피드백: '우리 모델만의 뛰어난 결과를 보여주는 예시가 와닿게 느껴지면 관심이 생길 것 같다.' → 개발자는 추상적 설명보다 실제 탐지 사례에 반응한다.
- 핵심 지적: 'Claude Code-Codex 같은 유명 에이전트에 URL 넣고 돌린 결과 대비 ScanOps 가 더 나은 예시를 보여줘야 한다. "보안 전용으로 파인튜닝했다"는 건 그렇구나 정도일 뿐, 시중 유료 플랜에 보안 검사를 시키는 것보다 효과가 좋은지를 사람들이 많이 따질 것이다.'
- 전환 포인트: '이것(범용 도구 대비 우위)만 좀 와닿게 느껴지면 관심이 많이 생길 것 같다.'
- 해커톤/제품화 관점: '제품화 시엔 그 우위 증명이 관건이지만, 해커톤용으로는 성능만 입증되면 아이디어가 괜찮다. 바이브 코딩과 보안이 트레이드오프라는 점에서 주제 자체가 핫하다.'

→ 인터뷰 ②의 지적은 본 보고서 §3.4.3.5(범용 LLM 은 놓치고 ScanOps 는 잡은 신규 CVE·멀티파일 사례)로 직접 대응했다. '보안 파인튜닝'이라는 설명이 아니라, 동일 입력에서 상용 모델이 놓친 취약점을 우리가 잡은 구체적 사례가 바로 그 '와닿는 증명'이다.

사람들의 인식과 ScanOps 의 필요성

- 인식 ① 개발자 대다수는 보안 지식이 낮다(자가평가 2~3/10). → 전문가 없이도 위험도·수정방법·참고문서를 떠먹여 주는 도구가 필요하며, ScanOps 의 CVSS·방어방침·레퍼런스 출력이 바로 그 공백을 메운다.
- 인식 ② 보안 점검의 필요성은 알지만 '속도 우선·설득 비용·내 지갑' 때문에 미룬다. → 가입 즉시 셀프서브로 쓰고 PR 마다 자동으로 도는 저마찰 구조가 'GitHub App'·Pro 무료체험으로 이 미루기를 깨는 열쇠다.
- 인식 ③ 회사 예산이면 100~200 만원도 지불 의향이 있다. → 개인 결제(월 2.9~9.9 만원)로 진입시킨 뒤 팀/회사 결제(B2B)로 확장하는 경로가 유효하다.
- 인식 ④ 개발도구에 월 14 만원, 개당 3 만원대를 이미 쓴다. → Pro 29,900 원은 기존 지출 습관 안에 있어 가격 저항이 낮다.
- 인식 ⑤ '코드·정보가 서버로 새지 않을까'가 실재하는 구매 장벽이다. → '코드 비전송·즉시 폐기'라는 ScanOps 의 핵심 차별점이 이 우려를 정면으로 해소해 구매 결정에 직접 작용한다.
- 인식 ⑥ '보안 전용 파인튜닝'이라는 말만으로는 약하다 — 범용 유료 도구(Claude Code·Codex·ChatGPT) 대비 실제 우위 사례가 있어야 지갑이 열린다. → §3.4.3.5 의 구체 사례가 이 설득의 핵심 무기다.
- 결정적 신호: 실제 사용 후 만족도가 급상승('개좋은데?!')하고 학습 가치까지 느꼈다. → 데모·무료체험(PLG)이 강력하며, 랜딩페이지의 실제 탐지 예시가 전환을 견인한다.

10.2 향후 고객 검증 계획

- 베타 전(3 개월): 스타트업 CTO·소규모팀 20~30 명 인터뷰. 질문 — '보안 점검을 미루는 이유?', 'Pro 29,900 원이면 쓸 의향은?', '코드 외부 전송 우려는?'
- 베타 후(6 개월): 대기자 100 명 확보, 유료 전환율·평균 사용 빈도·이탈률(Churn) 수집.

11. 리스크 관리 및 대응 전략

11.1 소유권 인증

- 웹 도메인: .well-known 파일 방식 — 도메인 루트에 검증 파일을 배치해 소유권을 확인(웹페이지에 절차 상세 안내 예정).
- GitHub 레포: GitHub App 인증 — 레포 첫 스캔 시 1 회 인증하고, 이후에는 재인증 없이 접근 권한을 유지.

11.2 오남용 방지

- DAST 남용 방지: 월 횟수 상한(Pro 5 회·Max 30 회), 타인 도메인 스캔 시 소유권 인증 필수.
- 사용량 한도: SAST-GitHub App 은 LOC 한도+줄 추가 구매로 통제해 서버 비용·남용을 동시에 관리.
- 법적 책임 범위: 이용약관에 '보안 진단 도구이며 진단 결과의 법적 효력은 없고 최종 책임은 사용자에게 있음' 명시. 주간 보안 요약(XLS) 리포트는 '내부 참고용'으로 포지셔닝.

11.3 데이터 보안

- 코드 처리: 분석 소스코드는 메모리에서만 처리 후 즉시 폐기, 삭제 로그로 증명.
- 결과 저장: 취약점 진단 결과만 저장(코드 원본 미저장), 사용자 요청 시 결과 삭제 가능.

11.4 기술 리스크 — 코드그래프/Neo4j 연동 지연

- 현재 그래프 판정 로직은 인메모리로 동작해 정확도엔 영향이 없으나, Neo4j(Aura) 연동·시각화·백엔드 DTO 확장이 지연될 경우 '그래프 시각화'를 제품 화면에서 보여주는 시점이 늦어질 수 있다.
- 대응: Aura 무료 티어로 우선 연동(저비용·단기간 가능), 백엔드 DTO 확장은 필드 3 개 추가 수준의 경량 작업으로 7.2 절 로드맵 0 단계에 우선 배정했다.